

Выбор модели

vlm-recipes · подзадача 1

1. Что на самом деле требуется

В архитектуре проекта **Skill** — это **механизм подстановки в контекст**, а не способность модели. Методику работы с гипотезой модель получает через `select_skill`, знать её заранее не обязана.

Значит, требования к модели узкие и измеримые:

1. точно исполняет инструкцию, поданную в контексте;
2. вовремя вызывает инструмент и вовремя *перестает* вызывать;
3. держит формат ответа, который разбирается программно;
4. пригодна к дообучению.

Предметные знания проверять не нужно — они приходят из Knowledge. Ресурсы под инференс и обучение здесь тоже не рассматриваются: они меняются от квартала к кварталу, а критерии — нет.

2. Почему семейство Qwen

Лицензия Apache 2.0. Без ограничений на дообучение, внедрение и распространение производных. У Llama и Gemma лицензии с оговорками, которые для внедрения существенны.

Вызов инструментов обучен, а не имитируется. В `chat_template` есть ветка по `tools`: описания функций подставляются в том формате, в котором модель их видела при обучении. У моделей без такой ветки вызов приходится просить текстом и разбирать вручную, надёжность заметно ниже.

Единая линейка. Текстовые и мультимодальные модели одного поколения имеют общий шаблон диалога и формат данных. Переход между размерами — замена одной строки; переход в другое семейство — переписывание коллатора и перепроверка всего заново.

Русский язык. Токенизация экономнее, чем у Llama, качество выше среднего по открытым моделям. На длинном контексте экономия токенов переходит прямо в память и деньги.

3. Размер: что он решает, а что нет

Размер задаёт *потолок* качества. Дообучение определяет, насколько модель к этому потолку приблизится на *вашей задаче*. Отсюда правило, которое стоит принять до сравнения моделей:

Дообученная модель на 7–9B на узкой задаче обгоняет модель на 70B без дообучения — почти всегда для правил поведения и формата, почти никогда для сложного многошагового рассуждения.

Класс	Роль
3–4B	судья в гардрейлах, классификация, отладка пайплайна. Не для основной роли: держит инструкцию хуже, чем нужно агенту
7–9B	рабочая модель для узкой предметной задачи после дообучения. Оптимум по соотношению качества к стоимости итерации
27–32B	заметный прирост на рассуждении и длинном контексте. Имеет смысл, когда 9B упёрлась в потолок качества, а не раньше
70B и выше	когда качество решает, а ресурсы есть. Дообучение только распределённое

Практический порядок: отладить пайплайн и данные на 3–4B, вести основную работу на 7–9B, подниматься выше только по результатам замера, а не априори.

4. Где смотреть: бенчмарки и дашборды

Общий балл на лидерборде почти бесполезен: он усредняет несравнимое. Для агента важны четыре узких свойства, и у каждого свой источник.

Свойство	Где мерить
вызывает инструмент, когда нужно	BFCL, категории simple, multiple, parallel
не вызывает, когда не нужно	BFCL, категория <i>irrelevance detection</i> — самая недооценённая
держит правила на длинной траектории	τ -bench, метрика pass^k
удерживает формат ответа	IFEval, режим strict

4.1. Вызов инструментов

BFCL — Berkeley Function Calling Leaderboard. <https://gorilla.cs.berkeley.edu/leaderboard.html>

Основной бенчмарк по вызову функций. Смотрите разбивку по категориям, а не итог: модели с одинаковым общим баллом различаются на *irrelevance* в полтора-два раза. Многоходовые сценарии — с версии v3.

τ -bench. <https://github.com/sierra-research/tau-bench>

Агент в предметной области с инструментами и политикой, которую нельзя нарушать под давлением пользователя. Ближе всего к постановке «student-in-the-loop с Policy». Метрика pass^k измеряет надёжность, а не удачу: смотрите $k = 4$, а не 1.

ToolBench, <https://github.com/OpenBMB/ToolBench>; **API-Bank;** **NexusRaven,** <https://huggingface.co/Nexusflow>.

Старше и частично насыщены, но полезны как второе мнение по формату вызовов и вложенным вызовам.

4.2. Следование инструкциям

IFEval. Инструкции с механически проверяемыми ограничениями: «ровно три абзаца», «ответ в JSON». Без судьи — проверка регулярным выражением; так же устроен наш `skills.jsonl`. Входит в Open LLM Leaderboard. Статья: <https://arxiv.org/abs/2311.07911>.

4.3. Агентские среды

AgentBench, <https://github.com/THUDM/AgentBench> — восемь сред, хорош для первого отсева. **GAIA,** <https://huggingface.co/spaces/gaia-benchmark/leaderboard> — вопросы с поиском и инструментами, три уровня сложности.

4.4. Мультимодальные модели

OpenVLM Leaderboard. https://huggingface.co/spaces/opencompass/open_vlm_leaderboard

Для документов смотрите DocVQA, ChartQA, OCRBench — не MMMU, который про общие знания.

4.5. Сводные дашборды

Дашборд	Ссылка	Чему верить
Open LLM Leaderboard	huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard	открытые модели, воспроизводимо, есть IFEval
LMarena	lmarena.ai	качество диалога по оценкам людей, не инструменты
Artificial Analysis	artificialanalysis.ai	скорость и цена по семействам
LiveBench	livebench.ai	обновляемые задания, меньше утечки
OpenCompass	rank.opencompass.org.cn	подробная разбивка по Qwen

Адреса по состоянию на середину 2026 года. Лидерборды переезжают, проверяйте.

5. Кто хорош в инструментах

Семейства, устойчиво занимающие верх BFCL среди открытых моделей по состоянию на середину 2026 года. Конкретные версии меняются каждые несколько месяцев, поэтому здесь семейства, а не модели.

Qwen — верх открытой части таблицы на протяжении нескольких поколений, вызов инструментов обучен нативно, есть все размеры.

Llama 3.x на 70B — сильна на BFCL, но токенизация русского хуже, а лицензия с оговорками.

GLM-4 и **DeepSeek-V3** — сильны в агентских сценариях, модели преимущественно крупные.

Mistral — хорошая поддержка инструментов, компактные модели, Apache 2.0.

Отдельный класс — модели, *специально* дообученные под вызов функций: **xLAM** (Salesforce), **Hermes** (Nous Research, на базе Llama), **ToolACE**, **Functionary** (MeetKai). На BFCL они обходят общие модели того же размера, иногда заметно.

Но у специализированных моделей есть цена: они слабее в свободном диалоге. Для агента, который не только вызывает инструменты, но и разговаривает со студентом, безопаснее общая модель с нативной поддержкой инструментов, чем узкий специалист по вызовам.

6. Проверка кандидата

`python tools/check_model.py <id>` — веса не качает, работает секунды. Четыре факта, которые нужно увидеть:

Поле	Требуемое значение
квантование	нет
вызов tools	обучен
русский	не меньше 3 символов на токен
лицензия	apache-2.0

Лидерборд отсекает заведомо слабые модели; между оставшимися выбирает собственный замер на своих данных и своём шаблоне — `tools/selfcheck.py` и ноутбуки.

7. Красные флаги

Только квантованные репозитории (-FP8, -NVFP4, -AWQ, -GPTQ). Обучать по ним нельзя. Хуже того, transformers не выдаёт ошибку, а начинает разжимать веса в bf16 — обнаруживается это по неожиданному расходу памяти. Признак в журнале: Decompressing.

Нет tools в шаблоне. Для агентского цикла модель годится плохо: формат вызова придётся изобретать и разбирать самому.

Неизвестный model_type. Установленный transformers старше модели. Обновить; если поддержки нет и в основной ветке — дообучение потребует ручной работы с кодом модели.

8. Открытая модель против API

Кандидаты в общей архитектуре проекта — закрытые модели по API. Для них дообучение недоступно или ограничено, а управление поведением сводится к системному промπτу.

Смысл ветки с открытой моделью не в том, чтобы догнать их по общему качеству, а в двух вещах, которых API не даёт: **закрепить Policy в весах** и **вмешаться в поведение на уровне активаций**. По этой оси её и следует оценивать.